

CSC316 – Homework #1

Password Hashing and Salting System

Student Name: Rayen Saliba

Student ID: 20231806

Course: CSC316

Homework: Homework #1

GitHub Repository: <https://github.com/RayenSaliba/Password-Hashing-and-Salting-using-python.git>

1. Introduction

This project demonstrates a basic password storage workflow using Python. The system generates a unique user identifier, creates a salt, applies SHA-256 hashing to the password, and derives a salted password value using PBKDF2-HMAC. The generated information is then organized into three in-memory tables for demonstration.

2. GitHub Repository

The complete source code for this homework is available in the following GitHub repository:
<https://github.com/RayenSaliba/Password-Hashing-and-Salting-using-python.git>

3. System Description

The program begins by requesting a username and password from the user. A UUID is then generated to provide a unique identifier for the user. The timestamp, username, and UUID are combined and processed with SHA-256 to produce the salt used by PBKDF2-HMAC.

The password is also hashed directly with SHA-256 for demonstration purposes. Finally, PBKDF2-HMAC with SHA-256 and 100,000 iterations is applied to the password using the generated salt. The resulting values are stored in three separate tables.

4. Password Hashing and Salting Process

Step	Description
1. User Input	The program receives a username and password from the user.
2. Unique User ID	A UUID4 value is generated using Python's uuid library.
3. Salt Generation	The timestamp, username, and UUID are combined and hashed with SHA-256 to produce salt bytes.
4. Direct Password Hash	The password is hashed with SHA-256 for comparison and demonstration.
5. PBKDF2-HMAC	The password is processed using PBKDF2-HMAC with SHA-256, the generated salt, and 100,000 iterations.
6. Table Storage	The username, user ID, hashes, and timestamp are placed into the three in-memory tables.

5. Tables Used by the Program

The program uses three Python lists as simple in-memory representations of tables.

Table	Stored Information	Purpose
Table 1	Username, Password, Timestamp	Demonstrates the original user input and registration time.

Table 2	Username, Unique User ID, Hashed Password	Stores the direct SHA-256 password hash.
Table 3	Username, Unique User ID, Hashed Salted Password	Stores the PBKDF2-HMAC derived password value.

6. Case Scenarios and Program Outputs

6.1 Case 1 – Normal User Registration

Enter a valid username and password and execute the program. The output should display the generated UUID, timestamp, SHA-256 password hash, and PBKDF2-HMAC derived password value inside the three tables.

```
Enter Username: oliver24
Enter Password: Ol!ver123&987

=====Table #1=====
{'Username': 'Oliver24', 'Password': 'Ol!ver123&987', 'Timestamp': '2026-09-20T22:19:02.097115'}

=====Table #2=====
{'Username': 'Oliver24', 'Unique User ID': '82c4c2cb-e67e-4745-abb6-35d76f8c1a11', 'Hashed Password': 'e948cec4c43f419b4f48fd65a7a9bbebb5ba2639ad8c2cd5e29d62f1821c1f7e'}

=====Table #3=====
{'Username': 'Oliver24', 'Unique User ID': '82c4c2cb-e67e-4745-abb6-35d76f8c1a11', 'Hashed Salted Password': 'c4ab13f5c82b4f68e96cbef09b6773d2e4252743b83687cc44b70e6289f60734'}
```

6.2 Case 2 – Same Password, Different User

Register another user using the same password. The generated user ID and salt-related values should differ because a different registration context produces different generated values.

```
Enter Username: Rayen12
Enter Password: Ol!ver123&987

=====Table #1=====
{'Username': 'Rayen12', 'Password': 'Ol!ver123&987', 'Timestamp': '2026-09-20T22:20:50.127532'}

=====Table #2=====
{'Username': 'Rayen12', 'Unique User ID': '8f531bae-5949-41bf-bcdc-105418e4e32d', 'Hashed Password': 'e948cec4c43f419b4f48fd65a7a9bbebb5ba2639ad8c2cd5e29d62f1821c1f7e'}

=====Table #3=====
{'Username': 'Rayen12', 'Unique User ID': '8f531bae-5949-41bf-bcdc-105418e4e32d', 'Hashed Salted Password': 'c26135a61e69968d867317258d28fd24cccd7b01accad48deca679af81fa894'}
```

6.3 Case 3 – Different Password

Register a user with a different password. The direct password hash and PBKDF2-HMAC derived value should change.

```

Enter Username: Rayen12
Enter Password: R@yS411b4

=====Table #1=====
{'Username': 'Rayen12', 'Password': 'R@yS411b4', 'Timestamp': '2026-09-20T22:22:38.142042'}

=====Table #2=====
{'Username': 'Rayen12', 'Unique User ID': '58eae16b-6c0e-4191-bd1e-7c24e9494baa', 'Hashed Password': '74eb55696a555f6499e6dae40399f96b999de6703463ee9eef67c02703777776'}

=====Table #3=====
{'Username': 'Rayen12', 'Unique User ID': '58eae16b-6c0e-4191-bd1e-7c24e9494baa', 'Hashed Salted Password': 'b89f351c9f8a2d59dd3c945b32bf55be48edf88686885944809b260b0effea8d'}

```

6.4 Case 4 – Repeated Execution

Run the program again with the same username and password. Because a new UUID and timestamp are generated, the salt input changes and the PBKDF2-HMAC derived value is expected to change as well.

```

Enter Username: Rayen12
Enter Password: R@yS411b4

=====Table #1=====
{'Username': 'Rayen12', 'Password': 'R@yS411b4', 'Timestamp': '2026-09-20T22:23:54.060119'}

=====Table #2=====
{'Username': 'Rayen12', 'Unique User ID': 'dc0b74b6-f7d5-47dc-9817-584081776628', 'Hashed Password': '5e5eb68fa091979d036f96c6574a019aa39b4531e2b6ce37de3e1769d7ac02e7'}

=====Table #3=====
{'Username': 'Rayen12', 'Unique User ID': 'dc0b74b6-f7d5-47dc-9817-584081776628', 'Hashed Salted Password': 'c804b0d36fbfdd24c4b9aeb6e85afa828c505fed0701a927718837814a6cc443'}

```

7. Security Considerations

- UUID4 provides a unique identifier for each registered user.
- The use of a salt means that password-derived values are not determined by the password alone.
- PBKDF2-HMAC intentionally performs many iterations, making password guessing more computationally expensive.
- The salt and the parameters required for password verification would need to be stored securely with the user record.
- The current program is an educational demonstration and should not be considered a complete production authentication system.

8. Limitations and Possible Improvements

- The current program stores the plaintext password in Table 1. A real authentication system should never store plaintext passwords.
- The salt is derived from application data. A production implementation would normally generate a cryptographically secure random salt independently using a secure random generator.

- The tables are Python lists stored only in memory, so all data is lost when the program terminates.
- The program does not currently implement login or password verification.
- Input validation and error handling could be added to make the system more robust.
- A production system should use a password-specific password hashing/KDF configuration and securely store the salt and required parameters.

9. Conclusion

This homework demonstrates the fundamental concepts of password hashing, salting, unique user identification, and key derivation using Python. The implementation shows the difference between a direct SHA-256 password hash and a PBKDF2-HMAC derived password value. The project also provides a foundation for extending the system with secure password verification, persistent storage, validation, and a complete authentication workflow.